

Uma Linguagem Funcional

(Reynolds, Theories of Programming Languages)

Maria João Frade

HASLab - INESC TEC
Departamento de Informática, Universidade do Minho

2025/2026

Uma Linguagem Funcional

Apresentam-se agora duas versões de uma linguagem funcional que estende o lambda calculus com operações lógicas e aritméticas, expressões condicionais, definições, tuplos, alternativas e listas.

- **FUNcbv** – Uma linguagem estrita, com uma semântica de avaliação aplicativa (“call-by-value”).
- **FUNcbn** – Uma linguagem não estrita, com uma semântica de avaliação normal (“call-by-name”).

FUNcbv

Uma Linguagem Funcional “call-by-value”

Sintaxe abstracta

$$\begin{aligned} \langle exp \rangle ::= & \langle var \rangle \mid \lambda \langle var \rangle . \langle exp \rangle \mid \langle exp \rangle \langle exp \rangle \\ & \mid 0 \mid 1 \mid 2 \mid \dots \mid \text{True} \mid \text{False} \\ & \mid \neg \langle exp \rangle \mid \neg \langle exp \rangle \mid \langle exp \rangle \mathbf{bop} \langle exp \rangle \\ & \mid \text{if } \langle exp \rangle \text{ then } \langle exp \rangle \text{ else } \langle exp \rangle \\ & \mid \langle \langle exp \rangle, \dots, \langle exp \rangle \rangle \mid \langle exp \rangle . \langle tag \rangle \\ & \mid @ \langle tag \rangle \langle exp \rangle \mid \text{sumcase } \langle exp \rangle \text{ of } (\langle exp \rangle, \dots, \langle exp \rangle) \\ & \mid \text{nil} \mid \langle exp \rangle :: \langle exp \rangle \mid \text{listcase } \langle exp \rangle \text{ of } (\langle exp \rangle, \langle exp \rangle) \\ & \mid \lambda \langle pat \rangle . \langle exp \rangle \mid \text{let } \langle pat \rangle \equiv \langle exp \rangle, \dots, \langle pat \rangle \equiv \langle exp \rangle \text{ in } \langle exp \rangle \\ & \mid \text{letrec } \langle var \rangle \equiv \lambda \langle pat \rangle . \langle exp \rangle, \dots, \langle var \rangle \equiv \lambda \langle pat \rangle . \langle exp \rangle \text{ in } \langle exp \rangle \end{aligned}$$
$$\mathbf{bop} \in \{+, -, *, \text{div}, \text{mod}, =, \neq, <, >, \leq, \geq, \vee, \wedge\}$$
$$\begin{aligned} \langle tag \rangle & ::= 1 \mid 2 \mid \dots \\ \langle pat \rangle & ::= \langle var \rangle \mid \langle \langle pat \rangle, \dots, \langle pat \rangle \rangle \end{aligned}$$

Precedências e associatividade dos operadores

- Para evitar o uso excessivo de parêntesis estipulamos a seguinte lista de precedências. A associatividade é à esquerda, excepto nos casos assinalados.

- .
- aplicação, @
- *, div, mod
- -, +
- =, ≠, <, >, ≤, ≥
- ¬
- ∧
- ∨
- :: (associativa à direita)

Formas canónicas

- A linguagem usa uma semântica de avaliação “call-by-value” (CBV).
- Serão dadas regras de inferência para a relação de avaliação $\Rightarrow_E$ (escreveremos abreviadamente $\Rightarrow$).
- $\Rightarrow$ relaciona as expressões com as formas canónicas da linguagem.
- Teremos **formas canónicas de diferentes tipos**

$$\begin{aligned} \langle cfm \rangle ::= & \lambda \langle var \rangle . \langle exp \rangle \\ & | \dots | -2 | -1 | 0 | 1 | 2 | \dots \\ & | \text{True} | \text{False} \\ & | \langle \langle cfm \rangle, \dots, \langle cfm \rangle \rangle \\ & | @ \langle tag \rangle \langle cfm \rangle \\ & | \text{nil} | \langle cfm \rangle :: \langle cfm \rangle \end{aligned}$$

Notação

- Avaliação CBV: $\langle exp \rangle \Rightarrow \langle cfm \rangle$
- Se a avaliação de uma expressão e não terminar ou bloquear, escreveremos simplesmente $e^\uparrow$.
- Na apresentação das regras da semântica CBV usaremos as seguintes **meta-variáveis**

e	$\langle exp \rangle_{\text{closed}}$	i	$\mathbb{Z}$
$\hat{e}$	$\langle exp \rangle$	k, n	$\mathbb{N}$
z	$\langle cfm \rangle$	b	$\mathbb{B}$
v, u	$\langle var \rangle$	p	$\langle pat \rangle$

- $[i]$ e $[b]$ denotam as formas canónicas das expressões i e b .
 - Ex: se i denota o inteiro 3 e i' o inteiro 2, $[i + i']$ denota o inteiro 5.

Semântica de avaliação “call-by-value”

- Formas canónicas**

$$\frac{}{z \Rightarrow z}$$

- Aplicação**

$$\frac{e \Rightarrow \lambda v. \hat{e} \quad e' \Rightarrow z' \quad \hat{e}[z'/v] \Rightarrow z}{e e' \Rightarrow z}$$

Semântica de avaliação “call-by-value”

• Operadores unários

$$\frac{e \Rightarrow [i]}{-e \Rightarrow [-i]}$$

$$\frac{e \Rightarrow [b]}{\neg e \Rightarrow [\neg b]}$$

Semântica de avaliação “call-by-value”

• Operadores binários

$$\frac{e_1 \Rightarrow [i_1] \quad e_2 \Rightarrow [i_2]}{e_1 \text{ bop } e_2 \Rightarrow [i_1 \text{ bop } i_2]} \text{ para bop} \in \{+, -, *, =, \neq, <, >, \leq, \geq\}$$

$$\frac{e_1 \Rightarrow [i_1] \quad e_2 \Rightarrow [i_2]}{e_1 \text{ bop } e_2 \Rightarrow [i_1 \text{ bop } i_2]} \text{ para bop} \in \{\text{div}, \text{mod}\} \text{ e } [i_2] \neq 0$$

$$\frac{e_1 \Rightarrow [b_1] \quad e_2 \Rightarrow [b_2]}{e_1 \text{ bop } e_2 \Rightarrow [b_1 \text{ bop } b_2]} \text{ para bop} \in \{\vee, \wedge\}$$

Semântica de avaliação “call-by-value”

• Expressões condicionais

$$\frac{e_1 \Rightarrow \text{True} \quad e_2 \Rightarrow z}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Rightarrow z}$$

$$\frac{e_1 \Rightarrow \text{False} \quad e_3 \Rightarrow z}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Rightarrow z}$$

Semântica de avaliação “call-by-value”

• Tuplos

$$\frac{e_1 \Rightarrow z_1 \quad \dots \quad e_n \Rightarrow z_n}{\langle e_1, \dots, e_n \rangle \Rightarrow \langle z_1, \dots, z_n \rangle}$$

$$\frac{e \Rightarrow \langle z_1, \dots, z_n \rangle}{e.k \Rightarrow z_k} \text{ para } k \in \{1, \dots, n\}$$

Semântica de avaliação “call-by-value”

• Alternativas

$$\frac{e \Rightarrow z}{@k e \Rightarrow @k z}$$

$$\frac{e \Rightarrow @k z \quad e_k z \Rightarrow z' \quad \text{para } k \in \{1, \dots, n\}}{\text{sumcase } e \text{ of } (e_1, \dots, e_n) \Rightarrow z'}$$

Semântica de avaliação “call-by-value”

• Listas

$$\frac{}{\text{nil} \Rightarrow \text{nil}}$$

$$\frac{e \Rightarrow z \quad e' \Rightarrow z'}{e :: e' \Rightarrow z :: z'}$$

$$\frac{e \Rightarrow \text{nil} \quad e_e \Rightarrow z}{\text{listcase } e \text{ of } (e_e, e_{ne}) \Rightarrow z}$$

$$\frac{e \Rightarrow z :: z' \quad e_{ne} z z' \Rightarrow z''}{\text{listcase } e \text{ of } (e_e, e_{ne}) \Rightarrow z''}$$

Exercício

Usando a semântica de avaliação *call-by-value*, calcule o valor de cada uma das seguintes expressões:

- ❶ $(\lambda u. \lambda l. \text{listcase } l \text{ of } (\text{nil}, \lambda h. \lambda t. u :: t)) (7+2) (1 :: 2 :: \text{nil})$
- ❷ $\langle \text{True}, \lambda x. x + x, 5 * 3 \rangle. 2 ((\lambda y. y + 1) 3)$
- ❸ $(\lambda f. \lambda u. \text{sumcase } (f u) \text{ of } (\lambda x. x, \lambda x. x * 2, \lambda x. 10)) (\lambda x. @2 x) ((\lambda y. y * y) 3)$

Açúcar sintático

Embora possam ser definidas regras de avaliação para as definições e padrões, é mais simples ver essas construções como açúcar sintático.

• Definições

$$\text{let } p_1 \equiv e_1, \dots, p_n \equiv e_n \text{ in } e \doteq (\lambda p_1. \dots \lambda p_n. e) e_1 \dots e_n$$

• Padrões

$$\lambda \langle p_1, \dots, p_n \rangle. e \doteq \lambda v. \text{let } p_1 \equiv v.1, \dots, p_n \equiv v.n \text{ in } e$$

Açúcar sintático

$$\begin{aligned}\text{let } \langle x, y \rangle \equiv w, z \equiv 2 * n \text{ in } x + y + z \\ \doteq (\lambda \langle x, y \rangle. \lambda z. x + y + z) w (2 * n) \\ \doteq (\lambda v. \text{let } x \equiv v.1, y \equiv v.2 \text{ in } \lambda z. x + y + z) w (2 * n) \\ \doteq (\lambda v. (\lambda x. \lambda y. \lambda z. x + y + z) (v.1) (v.2)) w (2 * n)\end{aligned}$$

Açúcar sintático

Uma definição alternativa para listas

- As listas podem ser vistas como estruturas de dados construídas à custa das alternativas e dos tuplos.
- Nesta abordagem uma lista é um valor alternativo que pode ser:
 - a etiqueta 1 seguida do tuplo vazio; ou
 - a etiqueta 2 seguida do par com a cabeça e cauda da lista.

$$\begin{aligned}\text{nil} &\doteq @1 \langle \rangle \\ e :: e' &\doteq @2 \langle e, e' \rangle \\ \text{listcase } e \text{ of } (e_e, e_{ne}) &\doteq \text{sumcase } e \text{ of } (\lambda \langle \rangle. e_e, \lambda \langle x, y \rangle. e_{ne} x y) \\ &\quad \text{com } x, y \notin \text{FV}(e_{ne})\end{aligned}$$

Recursividade

- Numa definição $\text{let } v \equiv e \text{ in } e'$ as ocorrências de v em e' são ligadas, mas eventuais ocorrências de v em e são livres, dado que

$$\text{let } v \equiv e \text{ in } e' \doteq (\lambda v. e') e$$

- Ou seja

$$\text{FV}(\lambda p. e) = \text{FV}(e) - \text{FV}(p)$$

$$\begin{aligned}\text{FV}(\text{let } p_1 \equiv e_1, \dots, p_n \equiv e_n \text{ in } e) \\ = \text{FV}(e_1) \cup \dots \cup \text{FV}(e_n) \cup (\text{FV}(e) - (\text{FV}(p_1) \cup \dots \cup \text{FV}(p_n)))\end{aligned}$$

- Portanto, numa definição $\text{let } v \equiv e \text{ in } e'$, v não pode representar uma função recursiva.

Recursividade

- A definição de funções recursivas é feita através da construção

$$\text{letrec } \langle var \rangle \equiv \lambda \langle pat \rangle. \langle exp \rangle, \dots, \langle var \rangle \equiv \lambda \langle pat \rangle. \langle exp \rangle \text{ in } \langle exp \rangle$$

Note como as restrições sintáticas apenas deixam definir recursivamente funções (isto é característico do CBV).

- Assim,

$$\begin{aligned}\text{FV}(\text{letrec } v_1 \equiv \lambda p_1. e_1, \dots, v_n \equiv \lambda p_n. e_n \text{ in } e) \\ = (\text{FV}(\lambda p_1. e_1) \cup \dots \cup \text{FV}(\lambda p_n. e_n) \cup \text{FV}(e)) - \{v_1, \dots, v_n\}\end{aligned}$$

Semântica de avaliação “call-by-value”

• Recursividade

$$\frac{(\lambda v_1. \dots \lambda v_n. e) (\lambda u_1. e_1^*) \dots (\lambda u_n. e_n^*) \Rightarrow z}{\text{letrec } v_1 \equiv \lambda u_1. e_1, \dots, v_n \equiv \lambda u_n. e_n \text{ in } e \Rightarrow z}$$

onde cada e_i^* representa $\text{letrec } v_1 \equiv \lambda u_1. e_1, \dots, v_n \equiv \lambda u_n. e_n \text{ in } e_i$ e $v_1, \dots, v_n \notin \{u_1, \dots, u_n\}$

Note que $(\lambda u_i. e_i^*)$ são formas canónicas. Portanto, os letrec's interiores não serão avaliados a não ser que a aplicação de $(\lambda u_i. e_i^*)$ a um argumento seja avaliada, ou seja, a não ser que a chamada recursiva seja avaliada.

Podemos ver $\text{letrec } v_1 \equiv \lambda u_1. e_1, \dots, v_n \equiv \lambda u_n. e_n \text{ in } e \Rightarrow z$ como a avaliação CBV da expressão e no contexto contendo as definições das funções $v_1, \dots, v_n$, mutuamente recursivas.

Exemplos

Alguns exemplos

- A função factorial

$\text{letrec fact} \equiv \lambda n. \text{if } n = 0 \text{ then } 1 \text{ else } n * \text{fact } (n-1) \dots$

- A concatenação de listas

$\text{letrec append} \equiv \lambda x. \lambda y. \text{listcase } x \text{ of } (y, \lambda h. \lambda t. h :: \text{append } t y) \dots$

- A função map

$\text{letrec map} \equiv \lambda f. \lambda l. \text{listcase } l \text{ of } (\text{nil}, \lambda h. \lambda t. f h :: \text{map } f t) \dots$

- A função foldr

$\text{letrec foldr} \equiv \lambda f. \lambda z. \lambda l. \text{listcase } l \text{ of } (z, \lambda h. \lambda t. f h (\text{foldr } f z t)) \dots$

Exercícios

Apresente a avaliação de $\text{letrec fact} \equiv \dots \text{ in } (\text{fact } 1)$ até à sua forma canónica.

Tendo definido funções de ordem superior, podemos definir novas funções utilizando definições não recursivas. Por exemplo:

$\text{let append} \equiv \lambda x. \lambda y. \text{foldr } (\lambda h. \lambda r. h :: r) y x \dots$
 $\text{let inc} \equiv \lambda l. \text{map } (\lambda x. x + 1) l \dots$

Apresente definições alternativas para as funções fact e map.

Exercícios

Defina as seguintes funções (com recursividade explícita)

- Valor absoluto de um inteiro.
- Comprimento de uma lista.
- Testar se uma lista de inteiros está ordenada.
- Fusão de listas ordenadas.